ЮЖНО-КАЗАХСТАНСКИЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИМЕНИ М. АУЭЗОВА

Кафедра «Вычислительная техника и программное обеспечение»

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 7

Тема работы

Ввод и вывод в Прологе

Обработка строк

Выполнил(-а): Көптілеу Б.

Группа: ИП-24-6тк

Проверил(-а): Мелдебекова С.

Шымкент 2026

Цель и организация работы

Цель работы — изучить организацию ввода и вывода в Пролог-программах, переключение потоков с помощью readdevice и writedevice, обработку файлов, строк и списков.

Среда программирования — Visual Prolog 5.0. Все 15 заданий объединены в одну программу с текстовым меню. Программный код и результаты выполнения приведены в редактируемом виде.

Порядок запуска

В главном меню вводится номер задания от 1 до 15. После выполнения программа возвращается к меню. Для завершения вводится 0. В задании 9 для возврата необходимо ввести только exit и нажать Enter.

Файловые задания автоматически создают демонстрационные данные в текущем рабочем каталоге. При повторном запуске соответствующие файлы перезаписываются. Действительные числа вводятся с десятичной точкой.

Структура листинга

Объявления domains и predicates приведены перед правилами clauses. Затем следуют меню, общие операции с файлами и строками, реализации заданий 1–15 и завершающая цель goal. Блоки составляют одну программу и используются совместно.

Общие операции со строками используются в заданиях 7, 10, 11 и 12. Предикаты smaller и larger из задания 5 также используются в задании 11. Слова разделяются пробелами и табуляцией; знаки препинания остаются частью слов. Сортировка учитывает регистр и кодировку.

Предикаты book, athlete и tennis_partner объявлены как nondeterm, поскольку допускают несколько решений. База книг в задании 13 содержит демонстрационные записи.

Оформление результатов

В разделах «Результат выполнения» приведены фрагменты предоставленного протокола запуска программы. Повторяющееся главное меню опущено. В задании 14 сохранены обе попытки ввода и полученные ответы программы.

Объявления типов и предикатов

```
domains
    file = f; g; h; t
    intlist = integer*
    charlist = char*
    wordlist = string*
predicates
    menu
    dispatch(integer)
    execute(integer)
    task1
    task2
    task3
    task4
    task5
    task6
    task7
    task8
    task9
    task10
    task11
    task12
    task13
    task14
    task15
    create_text(file, string, string)
    copy_chars(file)
    read_chars(file, charlist)
    write_chars(charlist)
    remove_penultimate(charlist, charlist)
    read_integers(intlist)
    write_integers(intlist)
    negative_numbers(intlist, intlist, integer)
    count_odd(intlist, integer)
    odd_value(integer, integer)
    smaller(integer, integer, integer)
    larger(integer, integer, integer)
    integer_extremes(intlist, integer, integer, integer, integer)
    save_extremes(intlist)
    real_totals(real, real)
    separator(char)
    split_words(string, wordlist)
    take_word(string, string, string)
    count_words(wordlist, integer)
    join_words(wordlist, wordlist, wordlist)
    calculate(integer, integer, string)
    normalize_spaces(string, integer, string)
    sentence_loop
    handle_sentence(string)
    sort_lex(wordlist, wordlist)
    insert_lex(string, wordlist, wordlist)
    length_extremes(wordlist, integer, integer, integer, integer)
    show_length_extremes(wordlist)
    read_file_words(wordlist)
    sort_length(wordlist, wordlist)
    insert_length(string, wordlist, wordlist)
    print_word_lengths(wordlist)
    nondeterm book(string, string, integer)
    show_books(wordlist)
    print_books(wordlist)
    nondeterm athlete(string, string)
    nondeterm tennis_partner(string, string)
    find_pairs(string)
    show_pairs(string, wordlist)
    print_pairs(string, wordlist)
    qualification(real, string, string)
    show_qualification(real)
```

Главное меню программы

```
clauses
    menu :-
        nl,
        write("========================================"), nl,
        write("       LABORATORY WORK No. 7"), nl,
        write("       INPUT, OUTPUT AND STRINGS"), nl,
        write("========================================"), nl,
        write(" 1. Merge two files"), nl,
        write(" 2. Negative numbers from file"), nl,
        write(" 3. Delete penultimate character"), nl,
        write(" 4. Count odd numbers"), nl,
        write(" 5. Maximum and minimum"), nl,
        write(" 6. Sum and product of real numbers"), nl,
        write(" 7. Count words in a string"), nl,
        write(" 8. Calculator"), nl,
        write(" 9. Replace repeated spaces"), nl,
        write("10. Sort words lexicographically"), nl,
        write("11. Minimum and maximum word lengths"), nl,
        write("12. Sort file words by length"), nl,
        write("13. Find two-volume books"), nl,
        write("14. Find tennis pairs"), nl,
        write("15. Student qualification"), nl,
        write(" 0. Exit"), nl,
        write("========================================"), nl,
        write("Select task: "),
        readint(Choice),
        dispatch(Choice).
    dispatch(0) :- !,
        write("Program finished."), nl.
    dispatch(Choice) :-
        execute(Choice),
        menu.
    execute(1) :- !, task1.
    execute(2) :- !, task2.
    execute(3) :- !, task3.
    execute(4) :- !, task4.
    execute(5) :- !, task5.
    execute(6) :- !, task6.
    execute(7) :- !, task7.
    execute(8) :- !, task8.
    execute(9) :- !, task9.
    execute(10) :- !, task10.
    execute(11) :- !, task11.
    execute(12) :- !, task12.
    execute(13) :- !, task13.
    execute(14) :- !, task14.
    execute(15) :- !, task15.
    execute(_) :-
        write("Invalid task number. Enter 0..15."), nl.
```

Общие операции с файлами

```
create_text(F, Filename, Text) :-
    openwrite(F, Filename),
    writedevice(F),
    write(Text),
    writedevice(screen),
    closefile(F).
copy_chars(F) :-
    eof(F), !.
copy_chars(F) :-
    readchar(C),
    write(C),
    copy_chars(F).
read_chars(F, []) :-
    eof(F), !.
read_chars(F, [C | Rest]) :-
    readchar(C),
    read_chars(F, Rest).
write_chars([]).
write_chars([C | Rest]) :-
    write(C),
    write_chars(Rest).
read_integers([]) :-
    eof(f), !.
read_integers([X | Rest]) :-
    readint(X),
    read_integers(Rest).
write_integers([]).
write_integers([X | Rest]) :-
    write(X), nl,
    write_integers(Rest).
```

Общие операции со строками

```
separator(' ').
separator('\t').
split_words("", []) :- !.
split_words(S, Words) :-
    frontchar(S, C, Rest),
    separator(C), !,
    split_words(Rest, Words).
split_words(S, [Word | Words]) :-
    take_word(S, Word, Rest),
    split_words(Rest, Words).
take_word("", "", "") :- !.
take_word(S, "", S) :-
    frontchar(S, C, _),
    separator(C), !.
take_word(S, Word, Rest) :-
    frontchar(S, C, Tail),
    take_word(Tail, WordTail, Rest),
    frontchar(Word, C, WordTail).
count_words([], 0).
count_words([_ | Rest], Count) :-
    count_words(Rest, TailCount),
    Count = TailCount + 1.
join_words([], Words, Words).
join_words([H | T], Words, [H | Result]) :-
    join_words(T, Words, Result).
```

Задание 1 Объединение файлов

Создать символьные файлы f и g. Записать в файл h сначала компоненты файла f, затем компоненты файла g с сохранением порядка.

Программный код

```
task1 :-
    nl,
    write("--- TASK 1 ---"), nl,
    create_text(f, "f1.txt", "Hello, "),
    create_text(g, "g1.txt", "Bagdaulet!"),
    openread(f, "f1.txt"),
    openread(g, "g1.txt"),
    openwrite(h, "h1.txt"),
    writedevice(h),
    readdevice(f),
    copy_chars(f),
    readdevice(g),
    copy_chars(g),
    readdevice(keyboard),
    writedevice(screen),
    closefile(f),
    closefile(g),
    closefile(h),
    write("Files f1.txt and g1.txt merged."), nl,
    write("Result in h1.txt:"), nl,
    openread(h, "h1.txt"),
    readdevice(h),
    copy_chars(h),
    readdevice(keyboard),
    closefile(h),
    nl.
```

Результат выполнения

```
Select task: 1

--- TASK 1 ---
Files f1.txt and g1.txt merged.
Result in h1.txt:
Hello, Bagdaulet!
```

Вывод программы — задание 1

Задание 2 Отрицательные числа

Создать файл f, содержащий целые числа. Переписать в файл g отрицательные числа и определить их количество.

Программный код

```
negative_numbers([], [], 0).
negative_numbers([X | Rest], [X | Result], Count) :-
    X < 0, !,
    negative_numbers(Rest, Result, TailCount),
    Count = TailCount + 1.
negative_numbers([_ | Rest], Result, Count) :-
    negative_numbers(Rest, Result, Count).
task2 :-
    nl,
    write("--- TASK 2 ---"), nl,
    create_text(f, "f2.txt", "12\n-5\n0\n-17\n8\n-3\n"),
    openread(f, "f2.txt"),
    readdevice(f),
    read_integers(Numbers),
    readdevice(keyboard),
    closefile(f),
    negative_numbers(Numbers, Negatives, Count),
    openwrite(g, "g2.txt"),
    writedevice(g),
    write_integers(Negatives),
    writedevice(screen),
    closefile(g),
    write("Source: ", Numbers), nl,
    write("Negative numbers: ", Negatives), nl,
    write("Count: ", Count), nl,
    write("Saved to g2.txt"), nl.
```

Результат выполнения

```
Select task: 2

--- TASK 2 ---
Source: [12,-5,0,-17,8,-3]
Negative numbers: [-5,-17,-3]
Count: 3
Saved to g2.txt
```

Вывод программы — задание 2

Задание 3 Удаление предпоследнего символа

Создать символьный файл t. Удалить из текста файла t предпоследний элемент. В программе элементом считается символ.

Программный код

```
remove_penultimate([], []) :- !.
remove_penultimate([X], [X]) :- !.
remove_penultimate([_, Y], [Y]) :- !.
remove_penultimate([X | Rest], [X | Result]) :-
    remove_penultimate(Rest, Result).
task3 :-
    nl,
    write("--- TASK 3 ---"), nl,
    create_text(t, "t3.txt", "ABCDE"),
    openread(t, "t3.txt"),
    readdevice(t),
    read_chars(t, Source),
    readdevice(keyboard),
    closefile(t),
    remove_penultimate(Source, Result),
    openwrite(t, "t3.txt"),
    writedevice(t),
    write_chars(Result),
    writedevice(screen),
    closefile(t),
    write("Before: "),
    write_chars(Source), nl,
    write("After: "),
    write_chars(Result), nl,
    write("File t3.txt updated."), nl.
```

Результат выполнения

```
Select task: 3

--- TASK 3 ---
Before: ABCDE
After: ABCE
File t3.txt updated.
```

Вывод программы — задание 3

Задание 4 Количество нечётных чисел

Создать файл f из натуральных чисел. Найти количество нечётных чисел.

Программный код

```
odd_value(X, 1) :-
    X mod 2 = 1, !.
odd_value(_, 0).
count_odd([], 0).
count_odd([X | Rest], Count) :-
    odd_value(X, Added),
    count_odd(Rest, TailCount),
    Count = TailCount + Added.
task4 :-
    nl,
    write("--- TASK 4 ---"), nl,
    create_text(
        f,
        "f4.txt",
        "1\n2\n3\n4\n5\n6\n7\n8\n9\n10\n"
    ),
    openread(f, "f4.txt"),
    readdevice(f),
    read_integers(Numbers),
    readdevice(keyboard),
    closefile(f),
    count_odd(Numbers, Count),
    write("Numbers: ", Numbers), nl,
    write("Odd numbers: ", Count), nl.
```

Результат выполнения

```
Select task: 4

--- TASK 4 ---
Numbers: [1,2,3,4,5,6,7,8,9,10]
Odd numbers: 5
```

Вывод программы — задание 4

Задание 5 Максимум и минимум

Создать файл f, содержащий целые числа. Записать в файл g максимальное и минимальное числа из файла f.

Программный код

```
smaller(A, B, A) :-
    A <= B, !.
smaller(_, B, B).
larger(A, B, A) :-
    A >= B, !.
larger(_, B, B).
integer_extremes([], Min, Max, Min, Max).
integer_extremes([X | Rest], CurrentMin, CurrentMax, Min, Max) :-
    smaller(CurrentMin, X, NextMin),
    larger(CurrentMax, X, NextMax),
    integer_extremes(Rest, NextMin, NextMax, Min, Max).
save_extremes([]) :- !,
    write("Source file is empty."), nl.
save_extremes([First | Rest]) :-
    integer_extremes(Rest, First, First, Min, Max),
    writedevice(g),
    write(Max), nl,
    write(Min), nl,
    writedevice(screen),
    write("Maximum: ", Max), nl,
    write("Minimum: ", Min), nl,
    write("Saved to g5.txt: maximum, then minimum."), nl.
task5 :-
    nl,
    write("--- TASK 5 ---"), nl,
    create_text(f, "f5.txt", "12\n-7\n35\n0\n-19\n8\n"),
    openread(f, "f5.txt"),
    readdevice(f),
    read_integers(Numbers),
    readdevice(keyboard),
    closefile(f),
    write("Numbers: ", Numbers), nl,
    openwrite(g, "g5.txt"),
    save_extremes(Numbers),
    closefile(g).
```

Результат выполнения

```
Select task: 5

--- TASK 5 ---
Numbers: [12,-7,35,0,-19,8]
Maximum: 35
Minimum: -19
Saved to g5.txt: maximum, then minimum.
```

Вывод программы — задание 5

Задание 6 Сумма и произведение

Создать файл, состоящий из действительных чисел. Найти их сумму и произведение.

Программный код

```
real_totals(0.0, 1.0) :-
    eof(f), !.
real_totals(Sum, Product) :-
    readreal(X),
    real_totals(TailSum, TailProduct),
    Sum = X + TailSum,
    Product = X * TailProduct.
task6 :-
    nl,
    write("--- TASK 6 ---"), nl,
    create_text(f, "f6.txt", "1.5\n2.0\n-3.0\n4.0\n"),
    openread(f, "f6.txt"),
    readdevice(f),
    real_totals(Sum, Product),
    readdevice(keyboard),
    closefile(f),
    write("Numbers: 1.5, 2.0, -3.0, 4.0"), nl,
    write("Sum: ", Sum), nl,
    write("Product: ", Product), nl.
```

Результат выполнения

```
Select task: 6

--- TASK 6 ---
Numbers: 1.5, 2.0, -3.0, 4.0
Sum: 4.5
Product: -36
```

Вывод программы — задание 6

Задание 7 Количество слов

Ввести символьную строку с терминала. Определить в ней количество слов.

Программный код

```
task7 :-
    nl,
    write("--- TASK 7 ---"), nl,
    write("Enter a string: "),
    readln(S),
    split_words(S, Words),
    count_words(Words, Count),
    write("Words: ", Words), nl,
    write("Word count: ", Count), nl.
```

Результат выполнения

```
Select task: 7

--- TASK 7 ---
Enter a string: bagdaulet is best
Words: ["bagdaulet","is","best"]
Word count: 3
```

Вывод программы — задание 7

Задание 8 Арифметический калькулятор

Написать простой калькулятор, выполняющий четыре арифметических действия над целыми числами, вводимыми с терминала.

Программный код

```
calculate(A, B, "+") :- !,
    Result = A + B,
    write("Result: ", Result), nl.
calculate(A, B, "-") :- !,
    Result = A - B,
    write("Result: ", Result), nl.
calculate(A, B, "*") :- !,
    Result = A * B,
    write("Result: ", Result), nl.
calculate(_, 0, "/") :- !,
    write("Error: division by zero."), nl.
calculate(A, B, "/") :- !,
    Result = (A * 1.0) / B,
    write("Result: ", Result), nl.
calculate(_, _, _) :-
    write("Error: unknown operation."), nl.
task8 :-
    nl,
    write("--- TASK 8 ---"), nl,
    write("Enter first integer: "),
    readint(A),
    write("Enter second integer: "),
    readint(B),
    write("Enter operation (+, -, *, /): "),
    readln(Operation),
    calculate(A, B, Operation).
```

Результат выполнения

```
Select task: 8

--- TASK 8 ---
Enter first integer: 5
Enter second integer: 3
Enter operation (+, -, *, /): +
Result: 8
```

Вывод программы — задание 8

Задание 9 Замена повторяющихся пробелов

Считывать с терминала предложения и выводить их в формате, в котором все группы идущих подряд пробелов заменены одиночными пробелами.

Программный код

```
normalize_spaces("", _, "") :- !.
normalize_spaces(S, 1, Result) :-
    frontchar(S, ' ', Rest), !,
    normalize_spaces(Rest, 1, Result).
normalize_spaces(S, 0, Result) :-
    frontchar(S, ' ', Rest), !,
    normalize_spaces(Rest, 1, Tail),
    frontchar(Result, ' ', Tail).
normalize_spaces(S, _, Result) :-
    frontchar(S, C, Rest),
    normalize_spaces(Rest, 0, Tail),
    frontchar(Result, C, Tail).
handle_sentence("exit") :- !.
handle_sentence(S) :-
    normalize_spaces(S, 0, Result),
    write("Result: ", Result), nl, nl,
    sentence_loop.
sentence_loop :-
    write("Enter a sentence (exit to return): "), nl,
    readln(S),
    handle_sentence(S).
task9 :-
    nl,
    write("--- TASK 9 ---"), nl,
    sentence_loop.
```

Результат выполнения

```
Select task: 9

--- TASK 9 ---
Enter a sentence (exit to return):
bagdaulet
Result: bagdaulet

Enter a sentence (exit to return):
exit
```

Вывод программы — задание 9

Задание 10 Лексикографическая сортировка

Ввести строку с терминала. Преобразовать её в список слов, упорядоченных лексикографически.

Программный код

```
sort_lex([], []).
sort_lex([Word | Rest], Sorted) :-
    sort_lex(Rest, SortedTail),
    insert_lex(Word, SortedTail, Sorted).
insert_lex(Word, [], [Word]) :- !.
insert_lex(Word, [H | T], [Word, H | T]) :-
    Word <= H, !.
insert_lex(Word, [H | T], [H | Result]) :-
    insert_lex(Word, T, Result).
task10 :-
    nl,
    write("--- TASK 10 ---"), nl,
    write("Enter words: "),
    readln(S),
    split_words(S, Words),
    sort_lex(Words, Sorted),
    write("Original: ", Words), nl,
    write("Sorted: ", Sorted), nl.
```

Результат выполнения

```
Select task: 10

--- TASK 10 ---
Enter words: bagdaulet came from a Atyrau before he was a child
Original:
["bagdaulet","came","from","a","Atyrau","before","he","was","a","child"]
Sorted:
["Atyrau","a","a","bagdaulet","before","came","child","from","he","was"]
```

Вывод программы — задание 10

Задание 11 Длины слов

Ввести строку с терминала. Вычислить минимальную и максимальную длины слов строки.

Программный код

```
length_extremes([], Min, Max, Min, Max).
length_extremes(
    [Word | Rest],
    CurrentMin,
    CurrentMax,
    Min,
    Max
) :-
    str_len(Word, Length),
    smaller(CurrentMin, Length, NextMin),
    larger(CurrentMax, Length, NextMax),
    length_extremes(Rest, NextMin, NextMax, Min, Max).
show_length_extremes([]) :- !,
    write("The string contains no words."), nl.
show_length_extremes([First | Rest]) :-
    str_len(First, Length),
    length_extremes(Rest, Length, Length, Min, Max),
    write("Minimum word length: ", Min), nl,
    write("Maximum word length: ", Max), nl.
task11 :-
    nl,
    write("--- TASK 11 ---"), nl,
    write("Enter a string: "),
    readln(S),
    split_words(S, Words),
    show_length_extremes(Words).
```

Результат выполнения

```
Select task: 11

--- TASK 11 ---
Enter a string: bag dau let
Minimum word length: 3
Maximum word length: 3
```

Вывод программы — задание 11

Задание 12 Сортировка слов файла по длине

Создать текстовый файл f. Преобразовать его в список слов, упорядоченных по длине, и вывести список на терминал.

Программный код

```
read_file_words([]) :-
    eof(f), !.
read_file_words(Words) :-
    readln(Line),
    split_words(Line, LineWords),
    read_file_words(RestWords),
    join_words(LineWords, RestWords, Words).
sort_length([], []).
sort_length([Word | Rest], Sorted) :-
    sort_length(Rest, SortedTail),
    insert_length(Word, SortedTail, Sorted).
insert_length(Word, [], [Word]) :- !.
insert_length(Word, [H | T], [Word, H | T]) :-
    str_len(Word, WordLength),
    str_len(H, HeadLength),
    WordLength <= HeadLength, !.
insert_length(Word, [H | T], [H | Result]) :-
    insert_length(Word, T, Result).
print_word_lengths([]).
print_word_lengths([Word | Rest]) :-
    str_len(Word, Length),
    write(Word, " : ", Length), nl,
    print_word_lengths(Rest).
task12 :-
    nl,
    write("--- TASK 12 ---"), nl,
    create_text(
        f,
        "f12.txt",
        "programming is very interesting\nI like prolog\n"
    ),
    openread(f, "f12.txt"),
    readdevice(f),
    read_file_words(Words),
    readdevice(keyboard),
    closefile(f),
    sort_length(Words, Sorted),
    write("Original words: ", Words), nl,
    write("Sorted words: ", Sorted), nl, nl,
    write("Word : length"), nl,
    print_word_lengths(Sorted).
```

Результат выполнения

```
Select task: 12

--- TASK 12 ---
Original words: ["programming","is","very","interesting","I","like","prolog"]
Sorted words: ["I","is","very","like","prolog","programming","interesting"]

Word : length
I : 1
is : 2
very : 4
like : 4
prolog : 6
programming : 11
interesting : 11
```

Вывод программы — задание 12

Задание 13 Двухтомники заданного автора

Определить среди перечня книг двухтомники заданного автора. Фамилия автора вводится с клавиатуры.

Программный код

```
book("Selected works", "Auezov", 2).
book("Collected stories", "Auezov", 2).
book("Novel collection", "Auezov", 4).
book("Poems", "Abai", 2).
book("Words of wisdom", "Abai", 1).
book("Historical novels", "Esenberlin", 3).
book("Selected prose", "Esenberlin", 2).
print_books([]).
print_books([Title | Rest]) :-
    write("- ", Title), nl,
    print_books(Rest).
show_books([]) :- !,
    write("No two-volume books found."), nl.
show_books(Books) :-
    write("Two-volume books:"), nl,
    print_books(Books).
task13 :-
    nl,
    write("--- TASK 13 ---"), nl,
    write("Authors: Auezov, Abai, Esenberlin"), nl,
    write("Enter author surname: "),
    readln(Author),
    findall(Title, book(Title, Author, 2), Books),
    show_books(Books).
```

Результат выполнения

```
Select task: 13

--- TASK 13 ---
Authors: Auezov, Abai, Esenberlin
Enter author surname: Auezov
Two-volume books:
- Selected works
- Collected stories
```

Вывод программы — задание 13

Задание 14 Пары теннисистов

По базе спортсменов определить возможные пары выбранного теннисиста с другими теннисистами. Имя вводится с клавиатуры.

Программный код

```
athlete("Bagdaulet", "tennis").
athlete("Aidar", "tennis").
athlete("Dana", "tennis").
athlete("Arman", "football").
athlete("Aruzhan", "tennis").
athlete("Dias", "boxing").
tennis_partner(Name, Other) :-
    athlete(Other, "tennis"),
    Other <> Name.
print_pairs(_, []).
print_pairs(Name, [Other | Rest]) :-
    write(Name, " - ", Other), nl,
    print_pairs(Name, Rest).
show_pairs(_, []) :- !,
    write("No other tennis players found."), nl.
show_pairs(Name, Partners) :-
    write("Possible pairs:"), nl,
    print_pairs(Name, Partners).
find_pairs(Name) :-
    athlete(Name, "tennis"), !,
    findall(Other, tennis_partner(Name, Other), Partners),
    show_pairs(Name, Partners).
find_pairs(_) :-
    write("The athlete is unknown or does not play tennis."), nl.
task14 :-
    nl,
    write("--- TASK 14 ---"), nl,
    write("Tennis: Bagdaulet, Aidar, Dana, Aruzhan"), nl,
    write("Other sports: Arman, Dias"), nl,
    write("Enter athlete name: "),
    readln(Name),
    find_pairs(Name).
```

Результат выполнения

```
Select task: 14

--- TASK 14 ---
Tennis: Bagdaulet, Aidar, Dana, Aruzhan
Other sports: Arman, Dias
Enter athlete name: Bagdaulet, Arman, Aruzhan
The athlete is unknown or does not play tennis.

Select task: 14

--- TASK 14 ---
Tennis: Bagdaulet, Aidar, Dana, Aruzhan
Other sports: Arman, Dias
Enter athlete name: Bagdaulet
Possible pairs:
Bagdaulet - Aidar
Bagdaulet - Dana
Bagdaulet - Aruzhan
```

Вывод программы — задание 14

Задание 15 Квалификация студента

Определить квалификацию по баллам Z: M при 80 ≤ Z ≤ 100; S при 60 ≤ Z < 80; B при 40 ≤ Z < 60; N при 0 ≤ Z < 40.

Программный код

```
qualification(Z, "M", "Magistr") :-
    Z >= 80,
    Z <= 100, !.
qualification(Z, "S", "Specialist") :-
    Z >= 60,
    Z < 80, !.
qualification(Z, "B", "Bakalavr") :-
    Z >= 40,
    Z < 60, !.
qualification(Z, "N", "Neudachnik") :-
    Z >= 0,
    Z < 40.
show_qualification(Z) :-
    qualification(Z, Code, Title), !,
    write("Qualification: ", Code, " - ", Title), nl.
show_qualification(_) :-
    write("Error: score must be between 0 and 100."), nl.
task15 :-
    nl,
    write("--- TASK 15 ---"), nl,
    write("80..100 : M - Magistr"), nl,
    write("60..<80 : S - Specialist"), nl,
    write("40..<60 : B - Bakalavr"), nl,
    write("0..<40  : N - Neudachnik"), nl,
    write("Enter score Z: "),
    readreal(Z),
    show_qualification(Z).
```

Результат выполнения

```
Select task: 15

--- TASK 15 ---
80..100 : M - Magistr
60..<80 : S - Specialist
40..<60 : B - Bakalavr
0..<40  : N - Neudachnik
Enter score Z: 60
Qualification: S - Specialist
```

Вывод программы — задание 15

Запуск общей программы

Следующий блок размещается в самом конце листинга после всех фактов и правил. Он запускает главное меню.

```
goal
    menu.
```

Заключение

Программа объединяет операции с символьными и числовыми файлами, обработку строк, сортировку списков и поиск решений в базе фактов. Переключение потоков позволяет использовать одинаковые предикаты чтения и записи при работе с клавиатурой, экраном и файлами.

Рекурсивные предикаты выполняют последовательный обход данных, a findall собирает результаты поиска в список. В соответствующих разделах отчёта приведены результаты запусков из предоставленного протокола.